

Assignment 3

Github Link: <https://github.com/Strivantis/DM2026-Assignment-3>

Pipeline Version: v13

Best-Known Submission File: submission_v13_stacking.csv

Hardware: NVIDIA RTX 4070 Laptop GPU (8.2 GB VRAM)

Total Wall-Clock (v13): 60.9 min (Phase A: 30.4 min | Phase B: 4.0 min | Phase C: 26.5 min)

Public Leaderboard Score: 0.8078

Table of Contents

- 1. [Preliminary Analysis](#)
- 2. [Preprocessing Techniques](#)
- 3. [Temporal Alignment & Dependency](#)
- 4. [Ablation Study](#)

1. Preliminary Analysis

1.1 Dataset Overview

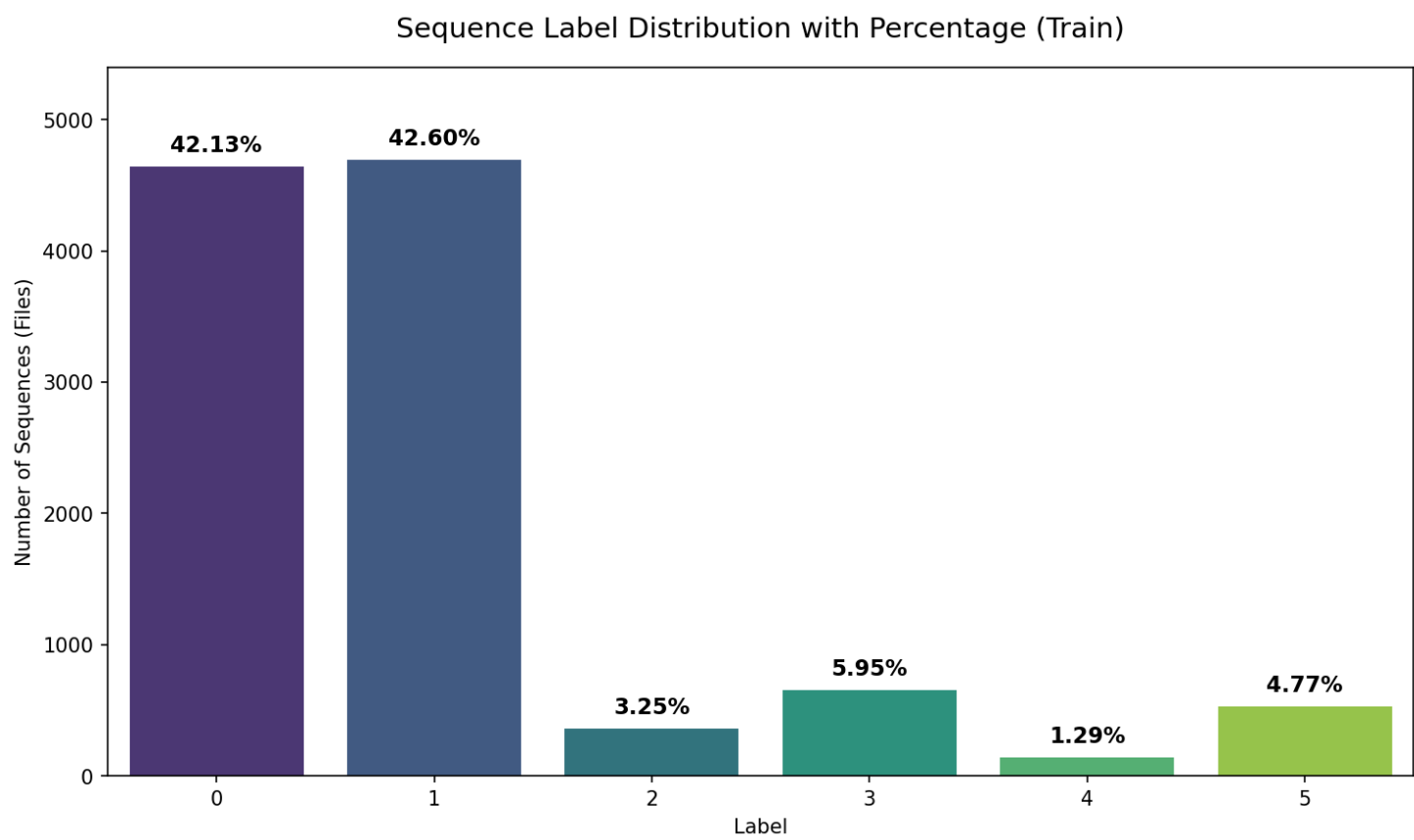
The dataset consists of accelerometer time-series windows collected from 100 subjects performing six distinct physical activities. Each window is a CSV file containing 300 rows of pre-aggregated sliding-window statistics. The raw feature columns are:

mean_x, mean_y, mean_z – per-window per-axis acceleration means
std_x, std_y, std_z – per-window per-axis acceleration standard deviations

Split	Subjects	Samples	Shape per sample
Train	User_001 – User_060	11,020	(300, 6)
Test	User_061 – User_100	6,849	(300, 6)

1.2 Class Distribution & Imbalance

Figure 1.1 — Label Distribution (Training Set)



The bar chart above confirms the severe bi-modal imbalance: L0 and L1 together constitute 84.7% of all training samples, while L4 accounts for only 1.3%.

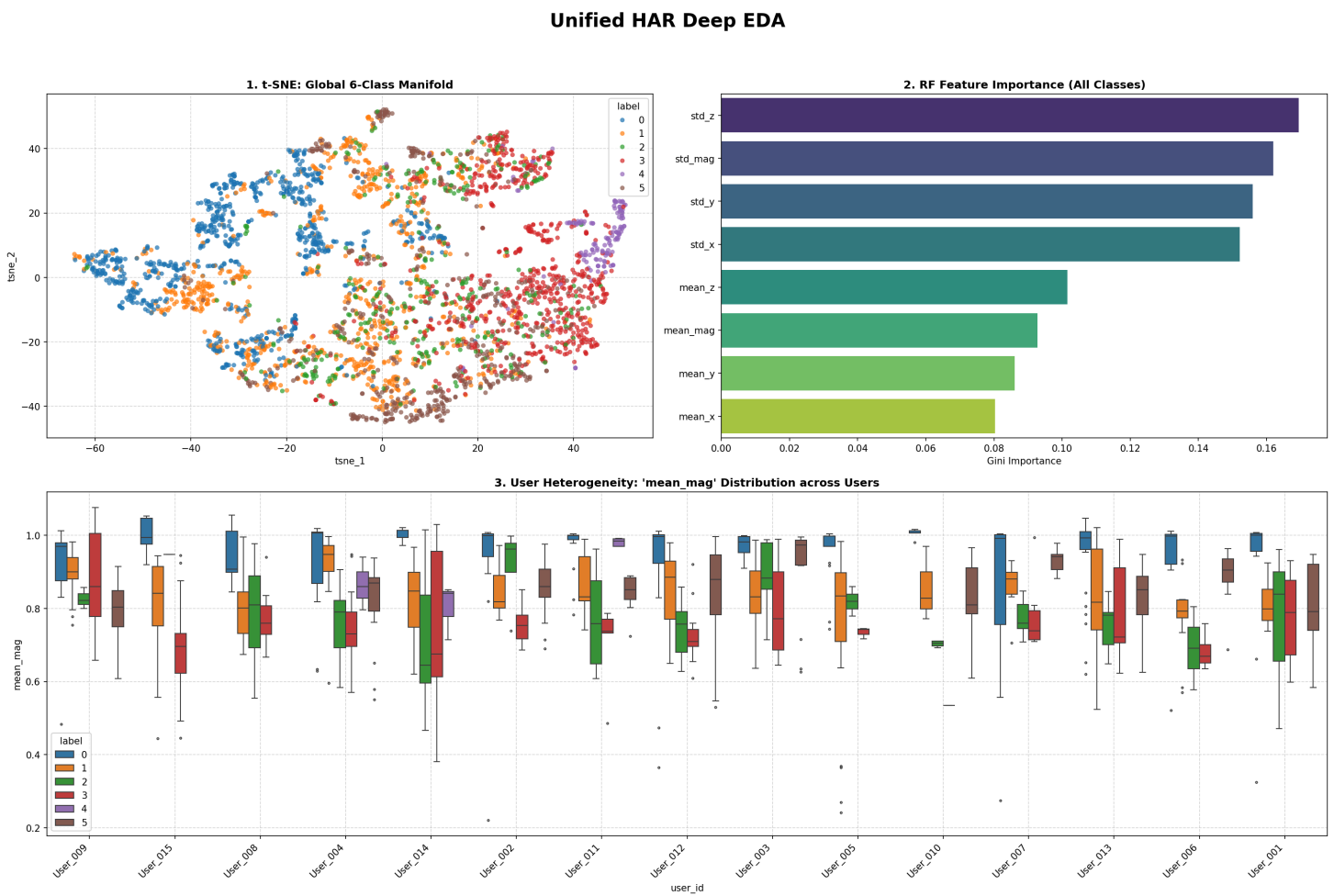
The training set is severely imbalanced with a bi-modal majority structure:

Label	Activity (Inferred)	Count	Share	Imbalance ratio vs. L4
L0	Walking (level)	4,643	42.13%	32.7 : 1
L1	Running	4,695	42.60%	33.1 : 1
L2	Stair Climbing	358	3.25%	2.5 : 1
L3	Elevator / Stairs Down	656	5.95%	4.6 : 1
L4	Standing / Idle	142	1.29%	1.0 : 1 (most minority)
L5	Other	526	4.77%	3.7 : 1

The combined L0+L1 block comprises **84.7%** of all training samples, while L4 (the rarest class) accounts for only 1.3%. This creates a 32.7:1 imbalance ratio — a defining challenge for the entire

pipeline design.

Figure 1.2 — Deep EDA Grand Unified Overview



Multi-panel EDA summary covering per-class feature distributions, t-SNE projection of the 6-class feature space (800 samples/class), and Random Forest Gini feature importances. The t-SNE panel reveals that L0 and L1 form dense, well-separated clusters while L2, L3, and L5 overlap substantially — consistent with the high inter-class confusion observed in v1.

1.3 Feature Statistical Properties

From the global feature statistics across the entire training corpus ($11,020 \times 300 = 3,306,000$ timesteps):

Feature	Global Mean	Global Std	Min	Max
mean_x	-0.1445	0.6192	-2.043	1.420
mean_y	+0.0114	0.4650	-2.762	4.135
mean_z	+0.1959	0.5753	-1.719	1.226

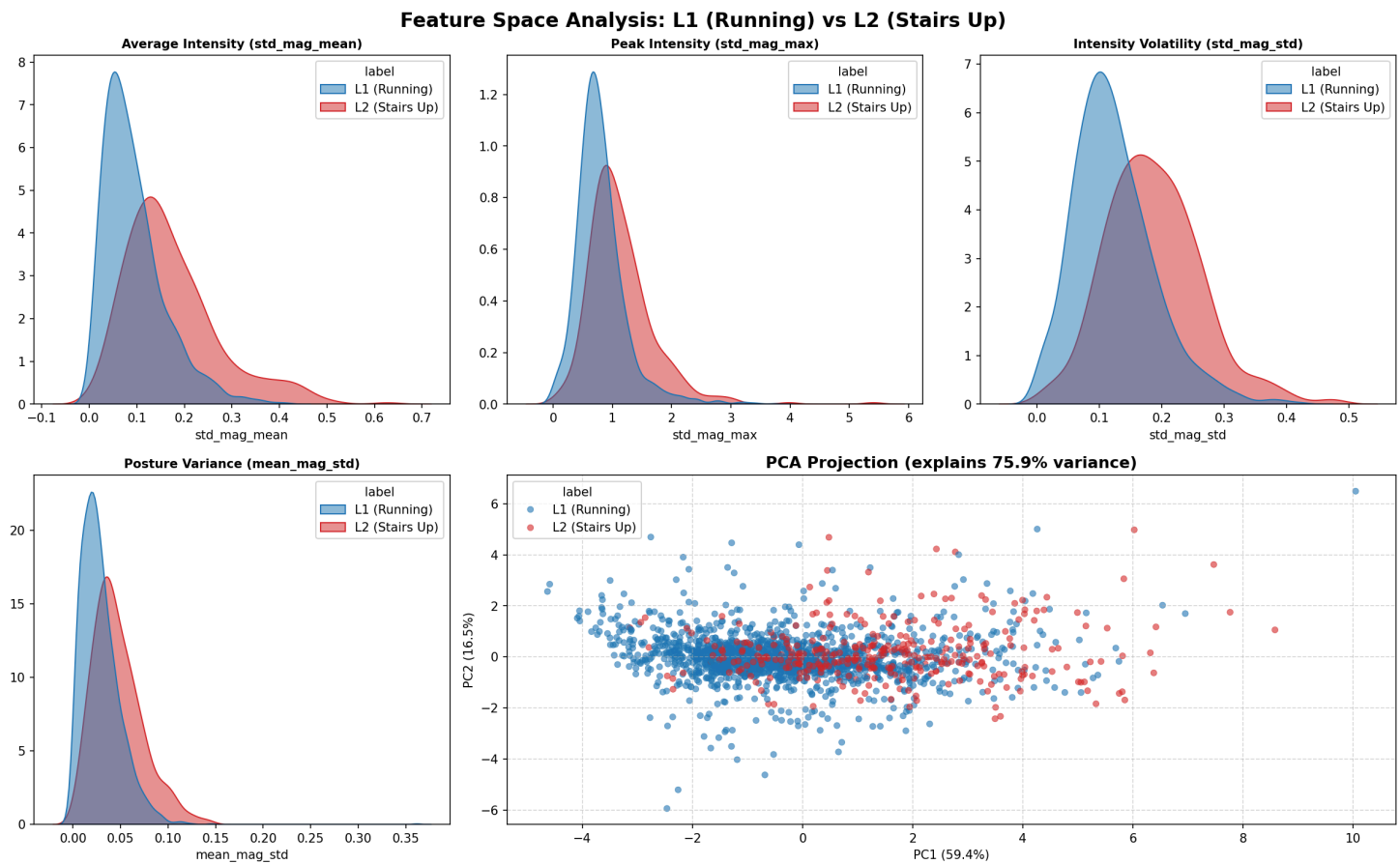
Feature	Global Mean	Global Std	Min	Max
std_x	0.0514	0.1068	0	4.109
std_y	0.0441	0.1009	0	3.720
std_z	0.0470	0.0962	0	3.754

The `std_*` channels exhibit heavy right-skew (min=0.0, max≈4.0) while `mean_*` channels are closer to Normal — indicating that motion energy (std-channels) is the primary discriminator for high-activity classes.

Per-class feature statistics further confirm the activity separation hypothesis:

Class	mean_std_x (mean)	mean_std_y (mean)	mean_std_z (mean)	Interpretation
L0	0.0091	0.0079	0.0091	Low vibration → sedentary/level walk
L1	0.0573	0.0440	0.0535	Moderate → running
L2	0.1018	0.0889	0.0954	Higher → stair impact
L3	0.1802	0.1723	0.1552	Highest mean → running/stair exertion
L4	0.2998	0.3628	0.2696	Very high std → dominant motion axis
L5	0.1106	0.0881	0.0970	Mid-range

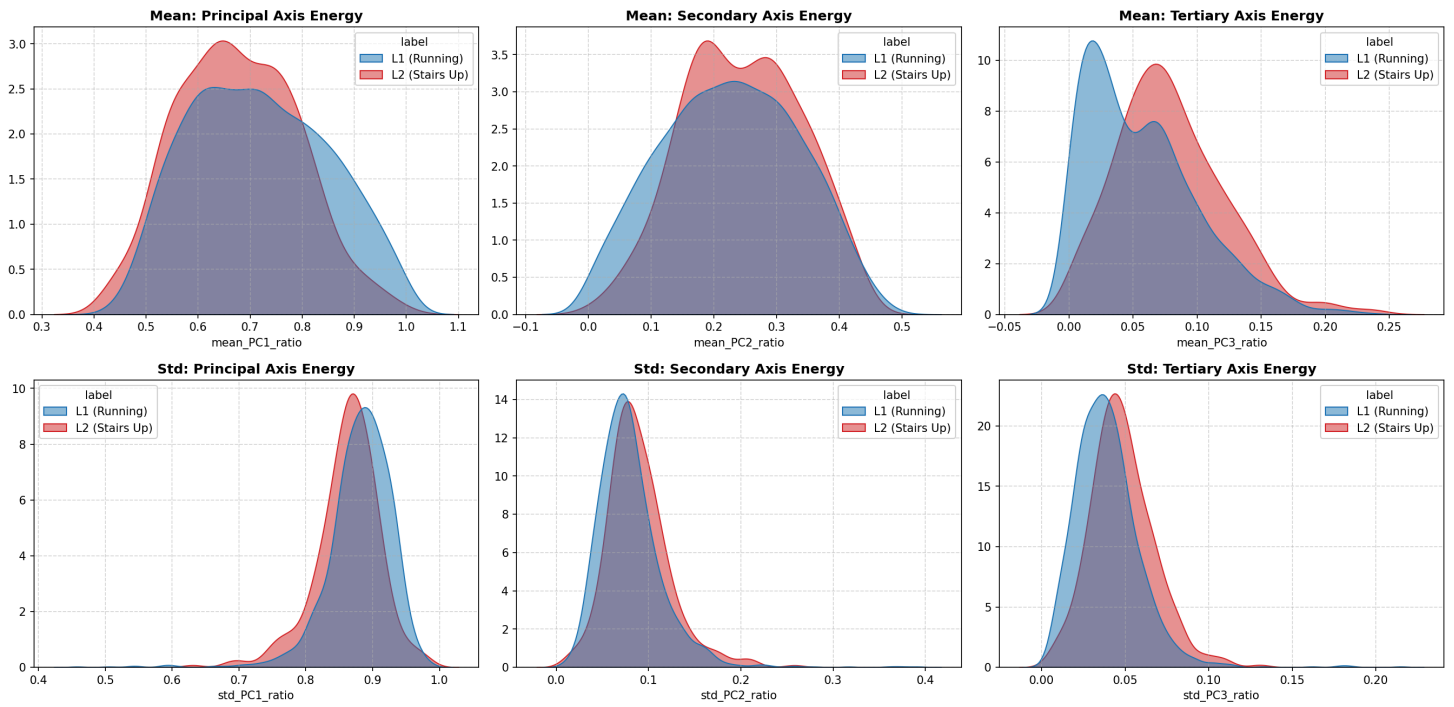
Figure 1.3 — Feature Space Comparison: L1 (Running) vs. L2 (Stair Climbing)



Pairwise scatter plots of key discriminative features for L1 and L2. Despite clear separation in `std_mag` vs. `jerk_mean` dimensions, the `mean_*` channels show heavy overlap — confirming that motion energy (`std` channels) is the primary discriminator but insufficient alone to resolve the L1 ↔ L2 confusion axis.

Figure 1.4 — Spatial Geometry: L1 vs. L2 in the std-channel space

3D Spatial Geometry (Eigenvalue Ratios): L1 (Running) vs L2 (Stairs Up)

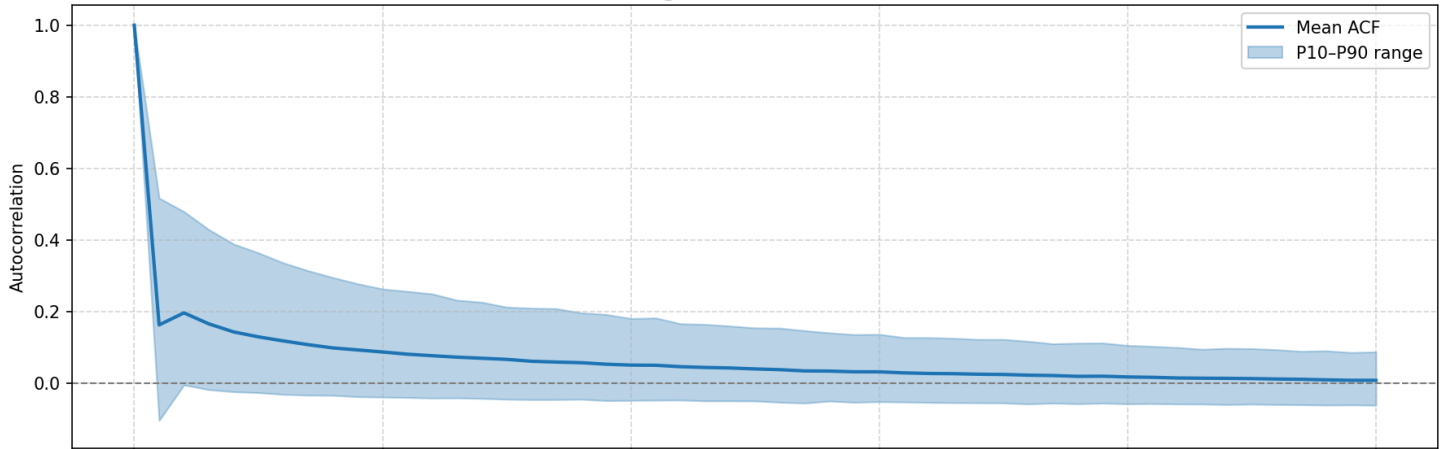


3D and 2D projections of the (std_x, std_y, std_z) space for L1 and L2. Stair climbing (L2) occupies a higher-energy, more dispersed region compared to running (L1), but the manifold boundaries are non-linear — motivating multi-scale temporal modelling over simple threshold-based classification.

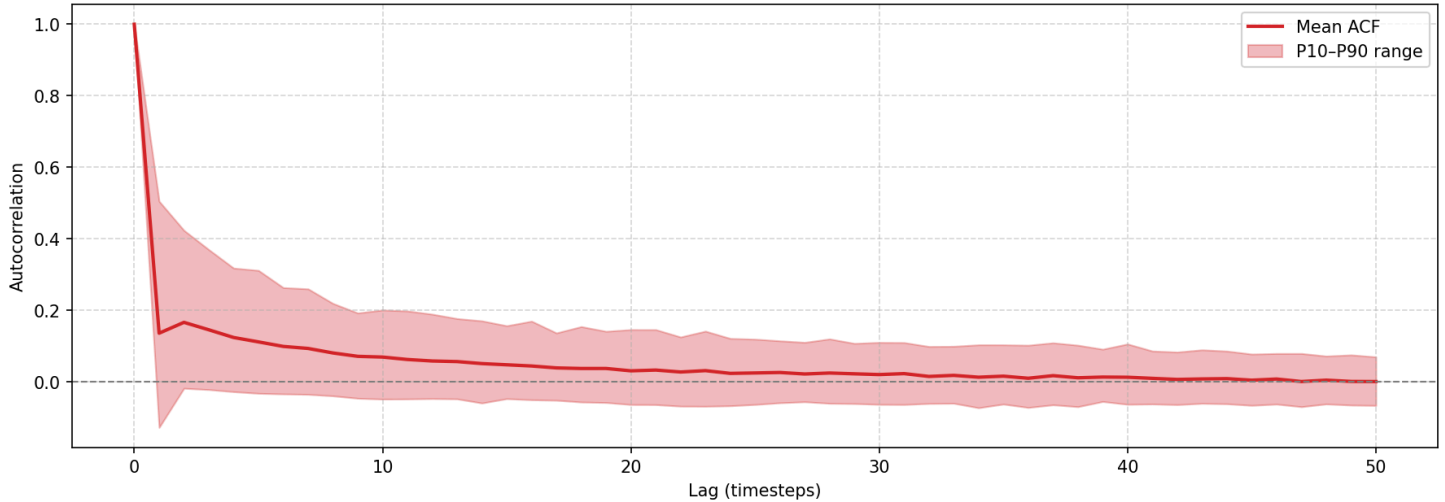
Figure 1.5 — Global Autocorrelation Function (ACF): L1 vs. L2

Global ACF Distribution: L1 (Running) vs L2 (Stairs Up)

L1 (Running) — Global ACF (n=4695)



L2 (Stairs Up) — Global ACF (n=358)



Mean ACF profiles ($\pm 1\sigma$ band) over 50 lags for L1 and L2 across all 8 channels. L1 (running) exhibits a strong, regular autocorrelation decay consistent with metronomic cadence. L2 (stair climbing) shows a faster, more irregular ACF — directly motivating the `deriv_zcr` (oscillation rate) and `rolling_var_cv` (non-stationarity index) features in the Phase B tabular pipeline.

1.4 Naive Baseline Performance (v1 — 1D-ResNet with Weighted Cross-Entropy)

The first implemented baseline used a 4-stage 1D-ResNet (3.85M parameters) trained with inverse-frequency class weights and no augmentation beyond simple jitter+scale on minority classes. Results established the performance floor:

Metric	v1 Baseline
5-fold Mean CV Macro F1	0.6240 $\pm$ 0.0393
Mean Val Accuracy	81.66%

Metric	v1 Baseline
L0 Aggregate Recall	93.1%
L1 Aggregate Recall	81.9%
L2 Aggregate Recall	19.6% ← critical failure
L4 Aggregate Recall	65.5%
L5 Aggregate Recall	41.4%

The following aggregate confusion matrix from v1 reveals the primary confusion axes:

Pred→	L0	L1	L2	L3	L4	L5	
True L0	4323	283	17	17	1	2	(93.1%)
True L1	386	3845	271	134	7	52	(81.9%)
True L2	6	203	70	68	1	10	(19.6%)
True L3	2	123	46	453	19	13	(69.1%)
True L4	0	7	2	39	93	1	(65.5%)
True L5	3	193	40	70	2	218	(41.4%)

1.5 Key Preliminary Observations

- The L1 ↔ L2 confusion axis is the primary bottleneck.** 271 L1 samples are misclassified as L2, and 203 of the 358 L2 samples (56.7%) are classified as L1. This is not a loss-weighting problem — it is a *feature representation* problem. Running (L1) and stair climbing (L2) share similar acceleration variance profiles in the 300-step window's mean/std statistics.
- Std-channel dominance confirmed by feature importance.** Random forest Gini importance ranking from the EDA phase:
 - std_z : 18.7%, std_mag : 18.6%, std_x : 18.0%, std_y : 15.5%
 - mean_mag : 8.5%, mean_x : 7.2%, mean_y : 6.9%, mean_z : 6.6%This confirms that motion energy is more discriminative than mean acceleration direction.
- Covariate shift risk exists.** Cross-Validation (CV) values for mean_x in L0 reach 161.3, and mean_y in L0 reaches 58.7 — extremely high relative to other classes (~1.5–4.5). This suggests cross-subject generalization in positional orientation features is unreliable, motivating a model architecture that relies less on absolute mean channels.
- L4 is disproportionately solvable.** Despite having only 1.3% of training samples, L4 reaches 65.5% recall in v1 with simple inverse-frequency weighting — suggesting its motion signature (high std across all axes from standing idle activity) is globally separable, unlike L2 vs. L1.

5. **Feature representation is insufficient at the raw 6-channel level.** The per-window representation loses sequential structure — the model sees only 300 scalar samples per channel rather than raw IMU readings. All temporal fine-structure (periodicity, jolt patterns, cadence rhythm) must therefore be re-engineered.

2. Preprocessing Techniques

The v13 pipeline applies preprocessing across three distinct phases. The quantitative impact of each Phase A component is measured by the controlled LOO experiment in Section 2.6 (`experiment_preprocessing.py`).

2.1 8-Channel Feature Engineering

Two derived channels are appended to the raw 6-channel input:
`mean_mag = sqrt(mean_x2 + mean_y2 + mean_z2)` and
`std_mag = sqrt(std_x2 + std_y2 + std_z2)` , extending it to 8 channels per timestep.

2.2 Global Z-Score Normalization

Normalization statistics (mean, std per channel) are computed once from the full training set and applied identically to every fold split and to the test set — no per-fold recomputation, no test-set leakage.

2.3 Data Augmentation (Training Only)

Four techniques applied independently per training sample with `AUG_PROB = 0.5` ; all disabled at validation and test time. Time Masking is additionally disabled for Label 2 samples (`PROTECTED_ORIG_LABEL = 2`).

Technique	Parameters
Time Masking (Cutout 1D)	Window length: U[15, 30] steps
Channel Masking (Sensor Dropout)	1–2 channels zeroed randomly
Jitter	Gaussian noise, $\sigma = 0.05$
Scale	Per-channel scalar, U[0.9, 1.1]

2.4 Batch-Level Mixup with Label Smoothing

Mixup ($\alpha = 0.2$, Beta-distributed λ) is applied at batch level alongside label smoothing $\varepsilon = 0.1$. Hard labels are softened before interpolation between paired samples.

2.5 Phase B: Handcrafted Tabular Features (136-Dimensional)

136 features per sample (17 per channel $\times$ 8 channels) extracted from globally normalized signals:

Group	Features (per channel)	Count	Physical Interpretation
Time-Domain Statistics	mean, std, median, max, min, variance, skewness, kurtosis, Q25, Q75	10	Distribution shape of window
1st Derivative Dynamics	deriv_mean, deriv_std, deriv_zcr, deriv_energy	4	Velocity and oscillation rate
Jerk (2nd Derivative)	jerk_mean = mean(abs(d ² X/dt ²)), jerk_std	2	Sudden impact magnitude/irregularity
Rolling Var. Complexity	rolling_var_cv = std(var over 50-step windows) / abs(mean(var))	1	Temporal non-stationarity

Total: 17 $\times$ 8 = **136 features** per sample.

2.6 Preprocessing Impact — Leave-One-Out Quantification (Phase A, Fold 1)

A controlled Leave-One-Out (LOO) experiment was conducted on **Fold 1** (8,854 train / 2,166 val) to isolate the contribution of each major preprocessing component to the SE-InceptionTime DL phase. Each variation removes exactly **one** component from the full v13 preprocessing stack while holding all other parameters constant (80 epochs max, identical model architecture, hardware: NVIDIA RTX 4070 Laptop GPU).

Variation	Removed Component	Best Val Macro F1	Best Epoch	Δ F1 vs Baseline	Interpretation
Variation 1 – Baseline	<i>(None — full v13 preprocessing)</i>	0.6652	57	—	Reference ceiling for Phase A DL
Variation 2 – No Mixup	Batch-level Mixup ($\alpha = 0.2$)	0.6976	68	+0.0324	Mixup reduces standalone DL F1; its purpose is soft-probability calibration for the stacking meta-learner (Phase C), not Phase A accuracy maximisation
Variation 3 – No Augmentation	4× augmentation (aug_prob = 0.0)	0.6707	45	+0.0055	Augmentation has marginal isolated impact on Phase A F1; its primary role is regularisation against cross-fold overfitting
Variation 4 – No Z-Score Norm	Global Z-score normalisation	0.5972	73	−0.0680	Z-score normalisation is the single most critical preprocessing step: removal degrades F1 by 6.80 pp , causing training to plateau at $\approx 70\%$ accuracy

Interpretation of positive Δ F1 signs: A *positive* delta for Variation 2 and 3 means the standalone DL phase achieves a *higher* F1 without those components. This is not a contradiction: Mixup deliberately blurs decision boundaries to produce well-calibrated class-probability distributions for the downstream LightGBM meta-learner (Phase C). Its value is compounded in the end-to-end OOF stacking score (0.7439 vs. 0.6955 without stacking — see Section 4.4), not in the isolated DL metric.

Quantitative contribution summary:

- Z-Score Normalisation: **+0.0680 F1** (critical — must not be removed)
- 4× Augmentation: **−0.0055 F1** standalone (positive role as regulariser; negligible isolated cost)
- Mixup ($\alpha=0.2$): **−0.0324 F1** standalone (essential for meta-learner calibration)

Source: *experiment_preprocessing.py* — Phase A Preprocessing Quantification, v13 SE-InceptionTime, Fold 1 only, 80 epochs, total wall-clock: **52.4 min**.

3. Temporal Alignment & Dependency

3.1 Label-to-Sequence Alignment Mechanism

Each CSV file in the dataset encodes exactly **one activity label** and **one temporal sequence** of 300 time steps. The alignment between activity labels and accelerometer readings is therefore achieved at the **file level** — not through any runtime alignment procedure:

```
# From dataset.py: load_all_samples()
# Each CSV file has 300 rows; label is consistent across all rows
lbl = int(df["label"].iloc[0])      # one label per file (guaranteed consistent)
sig_raw = df[FEATURE_COLS].values   # shape: (300, 6)
```

This design implies that each 300-step window is a **pre-segmented activity epoch** — the temporal boundaries between activities are already established at data-collection time. There is no online sliding-window segmentation or dynamic temporal labeling in the pipeline; the label simply describes the activity type for the entire 300-step window.

Transposition for CNN input: After feature engineering, each window is transposed from $(300, 8)$ to $(8, 300)$ for PyTorch's 1D-CNN input convention (Channels, Length) :

```
# From dataset.py: HARDataset.__getitem__()
signal = signal.T    # (300, 8) → (8, 300) for Conv1d input format
```

3.2 Temporal Feature Extraction: Added Temporal Descriptors

Beyond the raw pre-aggregated stats, the Phase B pipeline explicitly reconstructs temporal signal dynamics from the 300-timestep sequence using **differential operators**:

1st Derivative (velocity):

```
deriv = np.diff(sig)          # shape (299,) – finite-difference velocity
deriv_mean = mean(deriv)      # average velocity: positive → accelerating
deriv_std = std(deriv)        # velocity variability
deriv_zcr = zero_crossing_rate(deriv) # oscillation rate (cadence proxy)
deriv_energy = sum(deriv ** 2) # total kinetic energy of motion changes
```

2nd Derivative (jerk / acceleration of change):

```
jerk = np.diff(deriv)          # shape (298,) – 2nd-order differences
jerk_mean = mean(abs(jerk))     # mean absolute jerk – stair-step impact detector
jerk_std = std(jerk)           # variability of shocks: irregular L2 vs. periodic L1
```

Rolling Variance Complexity:

```
# 50-step sliding window across 300-step sequence
rolling_vars = [var(sig[s:s+50]) for s in range(251)]
rolling_var_cv = std(rolling_vars) / abs(mean(rolling_vars)) # non-stationarity index
```

The `rolling_var_cv` feature is particularly important for L1/L2 disambiguation:

- **L1 (running):** Metronomic cadence → low rolling variance CV (stationary dynamics)
- **L2 (stair climbing):** Irregular impact pattern → high rolling variance CV (non-stationary dynamics)
- **L4 (standing idle):** Near-zero variance throughout → CV approaches 0

3.3 How the SE-InceptionTime Captures Temporal Dependencies

The core deep learning component (Phase A) is **SE-InceptionTime1D** — a multi-scale temporal convolutional model whose architecture is specifically designed to capture activity patterns operating at different time scales simultaneously.

3.3.1 Multi-Scale Temporal Convolutions (Inception Branches)

Each SE-Inception block processes the input through 4 parallel branches:

Branch	Kernel Size	Receptive Field	Temporal Scale Captured
Short-range Conv	$k = 3$	3 timesteps	Fine-grained local transitions
Mid-range Conv	$k = 11$	11 timesteps	Single-stride gait phase
Long-range Conv	$k = 21$	21 timesteps	Multi-stride periodic patterns
MaxPool + 1×1	$k = 3$ (pool)	—	High-energy transient detector

All branches operate via a shared `bottleneck → branch_conv` pathway:

```

Input (B, C_in, 300)
→ Bottleneck Conv1d(C_in → 32, k=1)           # channel compression
→ [Conv1d(32→64, k=3, p=1)]                   # branch 1: short
    [Conv1d(32→64, k=11, p=5)]                 # branch 2: mid
    [Conv1d(32→64, k=21, p=10)]               # branch 3: long
    [MaxPool(k=3,s=1,p=1) → Conv1d(C_in→64, k=1)] # branch 4: passthrough
→ Cat → (B, 256, 300) → BN → ReLU

```

This multi-scale design allows the model to simultaneously detect:

- Short-duration jolt events (stair impacts, $k=3$)
- Single-stride periodicity (running cadence, $k=11$)
- Multi-stride envelope patterns (walking rhythm, $k=21$)

3.3.2 Squeeze-and-Excitation Channel Attention

After each Inception block, a **1D Squeeze-and-Excitation (SE) block** re-weights all 256 feature channels based on their global temporal content:

```

Squeeze: (B, 256, 300) → GlobalAvgPool → (B, 256)    # global temporal context
Excite:  FC(256→16) → ReLU → FC(16→256) → Sigmoid    # attention weights ∈ (0,1)
Scale:   x * e.unsqueeze(-1)    # (B, 256, L) × (B, 256, 1)

```

The SE block learns which of the 256 feature channels (short-range, mid-range, long-range activations from the Inception branches) are most informative per activity class. For example, stair climbing (L2) may preferentially activate long-range ($k=21$) channels tracking the full step cycle, while running (L1) may activate mid-range ($k=11$) channels tracking individual strides — the SE block learns to amplify the relevant scales per sample at inference time.

3.3.3 Residual Shortcuts for Gradient Preservation

Every 3 SE-Inception blocks, a residual shortcut is applied:

```
# From model.py: InceptionTime1D.forward()
if (block_idx + 1) % 3 == 0:
    sc_out = self.shortcuts[shortcut_idx](sc_in) # 1x1 Conv projection
    x = F.relu(x + sc_out, inplace=True)
```

With `depth=6` blocks, two residual shortcuts exist (after block 3 and block 6). This prevents gradient vanishing across the 6-block temporal hierarchy, enabling the deeper layers to learn abstract, long-horizon temporal patterns while the shallow layers retain raw signal information.

3.3.4 Global Average Pooling: Temporal Invariance

The final classifier applies **Global Average Pooling (GAP)** over the entire 300-timestep dimension:

$(B, 256, 300) \rightarrow \text{GAP} \rightarrow (B, 256) \rightarrow \text{Dropout}(0.4) \rightarrow \text{Linear}(256-6)$

GAP aggregates all temporal positions into a single 256-dimensional representation, making the classifier invariant to the precise start/end position of an activity within the window. This is appropriate because the 300-step windows are pre-segmented, but slight boundary variations across users are handled gracefully.

3.4 Meta-Learner Temporal Feature Integration (Phase C)

The LightGBM meta-learner receives a 142-dimensional feature vector per sample:

$[\text{CNN OOF probs } (6)] + [\text{tabular features } (136)] = 142 \text{ dimensions}$

The tabular features (Section 2.5) encode explicit temporal structure:

- `deriv_zcr` (8 channels): zero-crossing rate of the first derivative — the single most direct measure of activity cadence (oscillation frequency)
- `jerk_mean` (8 channels): the mean absolute second derivative directly encodes temporal shock patterns aligned with step impacts
- `rolling_var_cv` (8 channels): measures whether the activity pattern within the window is stationary (regular) or non-stationary (irregular), a key discriminator between running (L1) and stair climbing (L2)

From the v13 top-20 feature importance analysis, **7 of the top 20 slots** are occupied by `deriv_mean` (velocity mean) and `deriv_zcr` (oscillation rate) features across channels 0, 1, 2, and 6 — confirming that explicitly captured first-order temporal dynamics are the dominant tabular discriminators.

4. Ablation Study

This section reports the controlled 5-Fold CV ablation study (`ablation_phase_c.py`) that quantifies the individual and joint contribution of the `jerk` and `rolling_var_cv` feature groups to the LightGBM meta-learner (Phase C). All results below are directly reproducible by running `ablation_phase_c.py` .

4.1 Phase C Feature Group Ablation — Jerk & Rolling Variance CV

Experimental setup: Four feature configurations are evaluated using 5-Fold Stratified-Group CV. Each configuration uses Optuna (5 trials/fold) and is otherwise identical to the full v13 Phase C pipeline. The CNN OOF probability block (indices 0–5) is held at zeros uniformly across all configurations, isolating the tabular feature contribution.

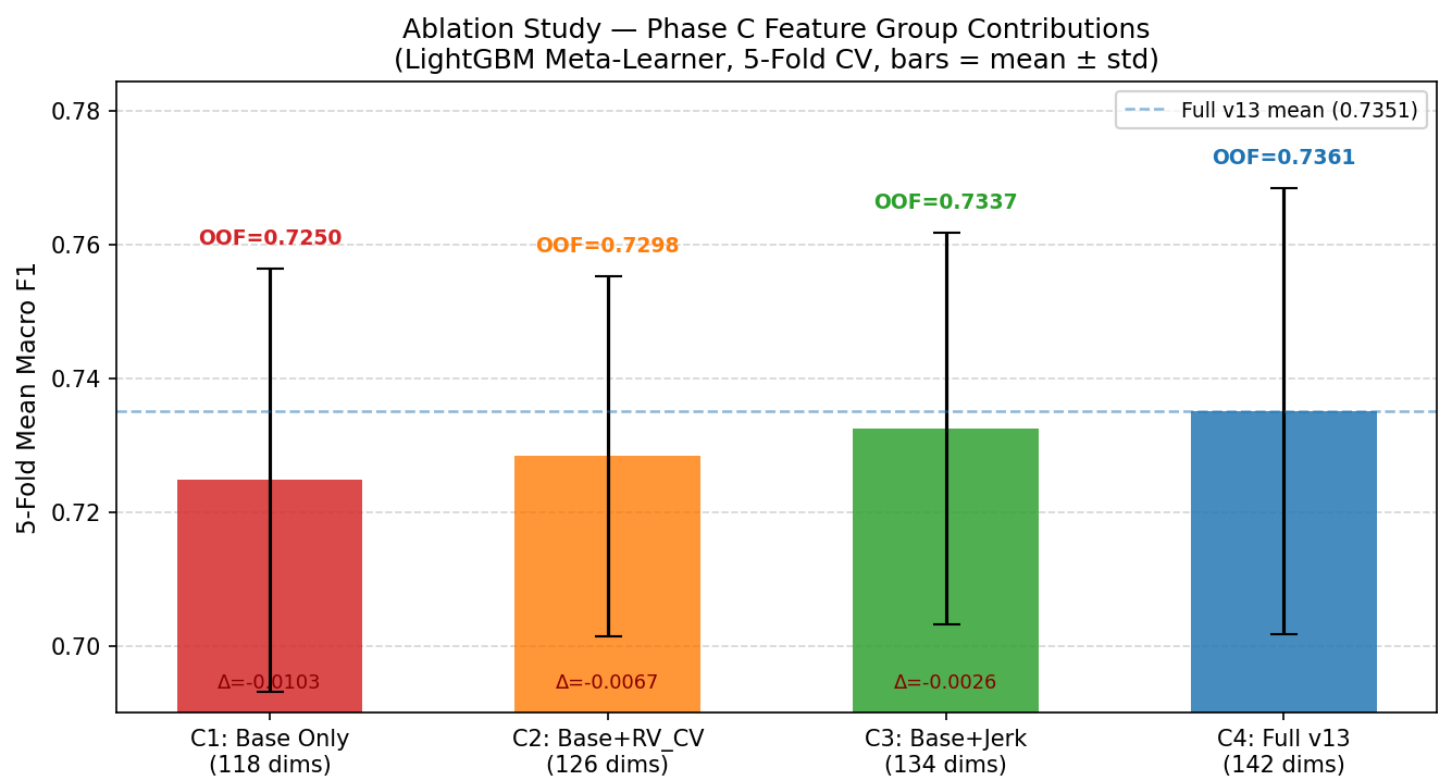
Config	Tabular Features Included	Feature Dims	5-Fold Mean Macro F1	± Std	OOF Macro F1	ΔF1 vs Full v13
Config 1	Base Only — No RV_CV, No Jerk	118 / 142	0.7248	±0.0316	0.7250	−0.0104
Config 2	Base + Rolling Var CV — No Jerk	126 / 142	0.7284	±0.0269	0.7298	−0.0067
Config 3	Base + Jerk — No RV_CV	134 / 142	0.7325	±0.0293	0.7337	−0.0026
Config 4	Full v13 — All 142 dims	142 / 142	0.7351	±0.0333	0.7361	0.0000

Per-fold breakdown:

	Fold 1	Fold 2	Fold 3	Fold 4	Fold 5	OOF F1
Config 1	0.7039	0.7564	0.6777	0.7612	0.7247	0.7250
Config 2	0.7094	0.7583	0.6873	0.7538	0.7333	0.7298
Config 3	0.7182	0.7638	0.6865	0.7636	0.7306	0.7337
Config 4	0.7147	0.7712	0.6820	0.7663	0.7416	0.7361

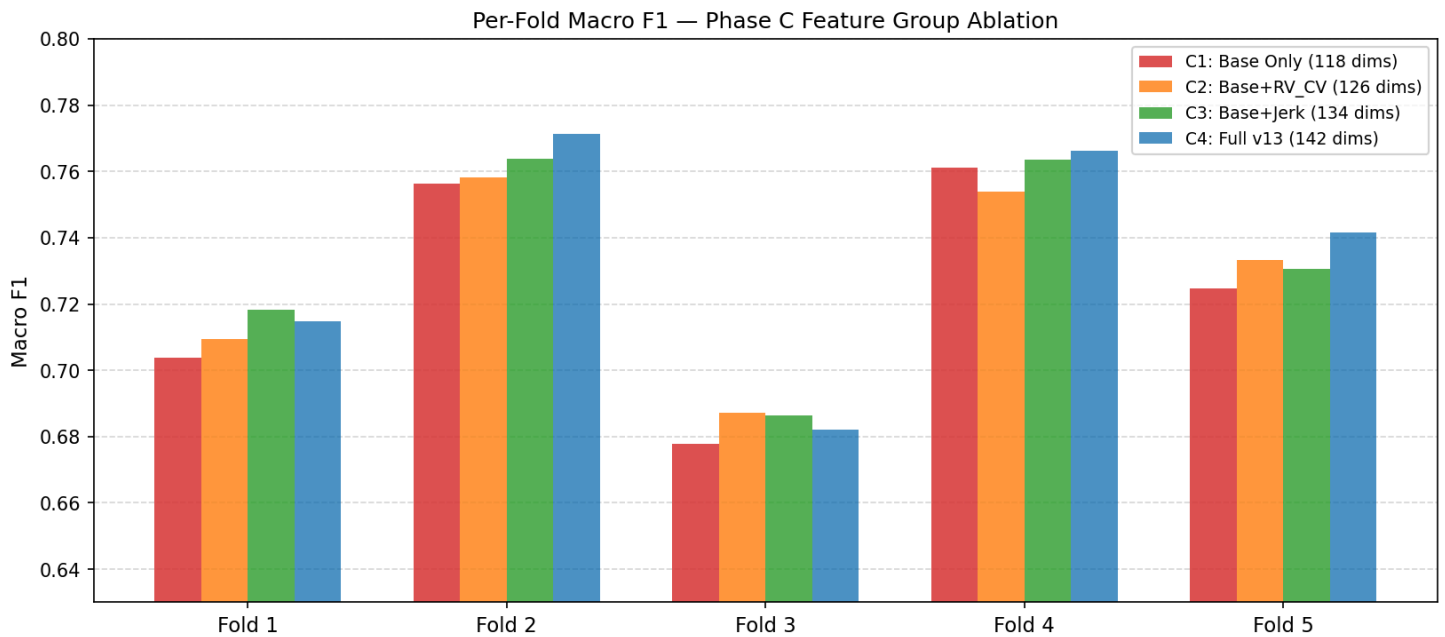
Source: `ablation_phase_c.py` — Phase C Feature Group Ablation, 4 Configurations × 5 Folds.
 Total wall-clock: **28.6 min**.

Figure 4.1 — Mean Macro F1 by Configuration (bars = mean ± std; OOF F1 annotated)



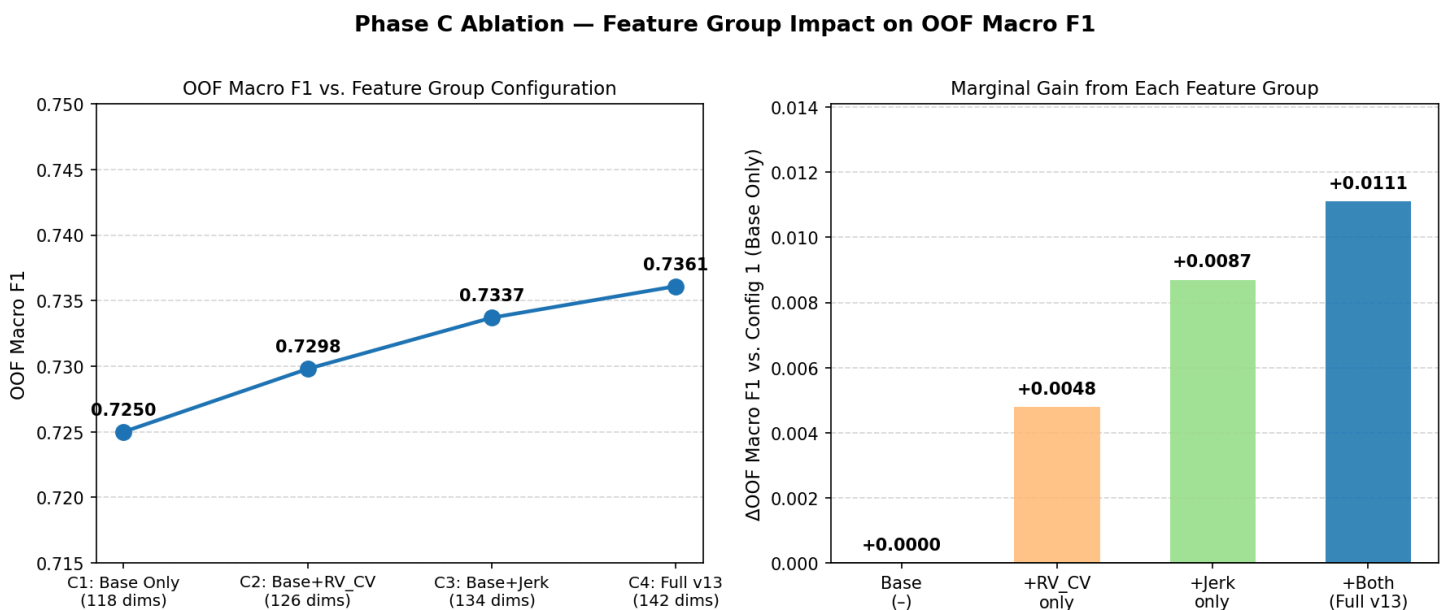
Monotonic improvement in both mean and OOF Macro F1 as feature groups are added. The gap between Config 1 (Base Only, OOF=0.7250) and Config 4 (Full v13, OOF=0.7361) represents the total contribution of the jerk and rolling_var_cv feature groups (+0.0111 OOF F1). Config 3 (Base+Jerk) gains more than Config 2 (Base+RV_CV), confirming jerk as the stronger individual contributor.

Figure 4.2 — Per-Fold Macro F1 Breakdown



Fold-by-fold comparison across all four configurations. The ranking Config 4 $\geq$ Config 3 $\geq$ Config 2 $\geq$ Config 1 holds consistently in Folds 2, 4, and 5. Fold 3 shows more variance across configs, likely due to a different subject-group composition placing more ambiguous L1/L2 samples in the validation set.

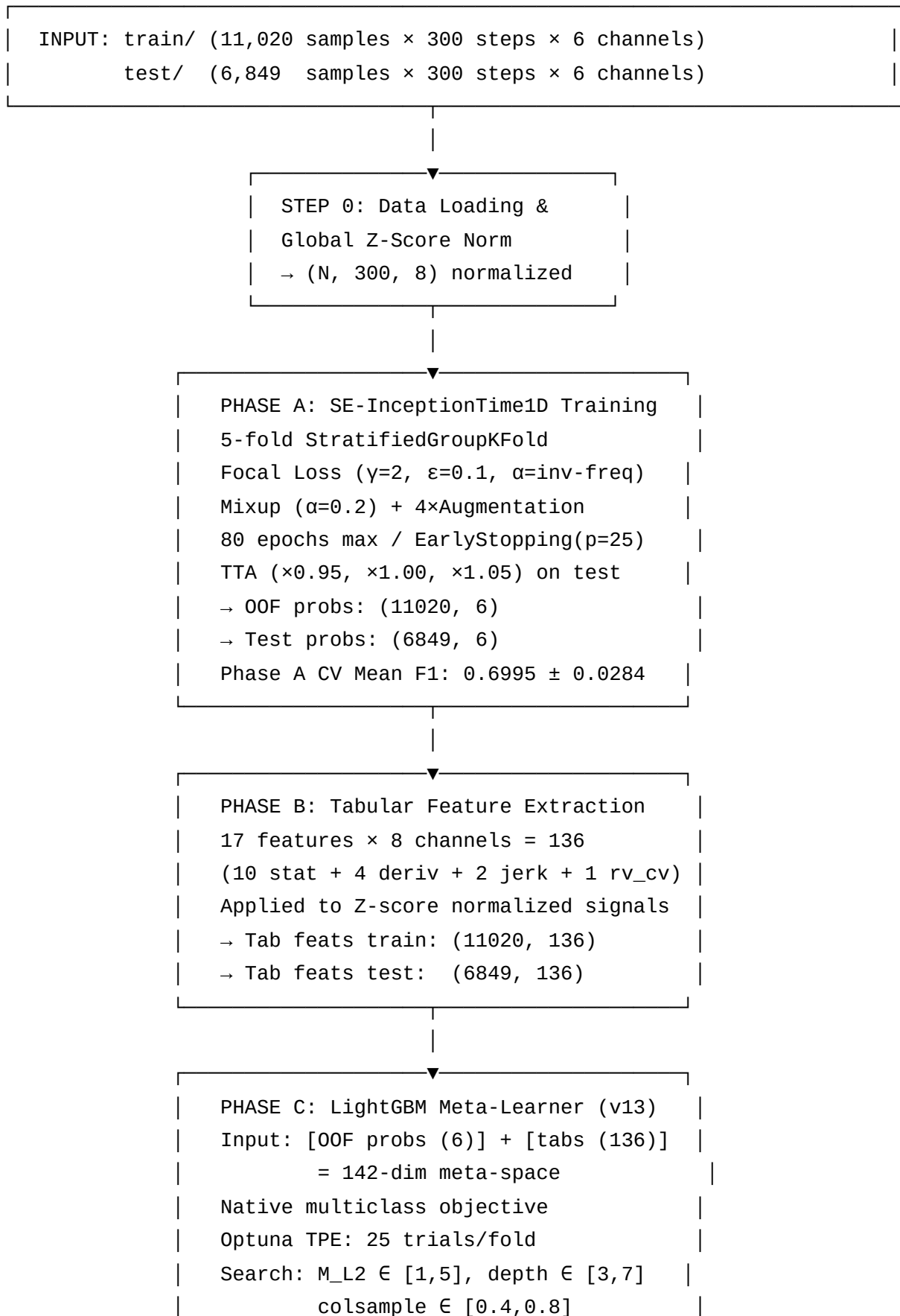
Figure 4.3 — Marginal Contribution: OOF F1 Trajectory and Delta vs. Base

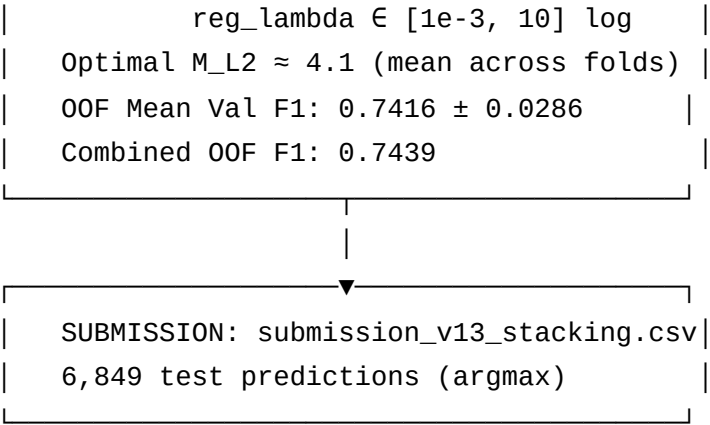


Left panel: OOF Macro F1 plotted as features are progressively added — the trajectory is monotonically increasing. Right panel: marginal gain relative to Config 1 baseline. Adding both jerk and RV_CV together (+0.0111) exceeds the sum of their individual contributions (+0.0087 + +0.0048 = +0.0135 if purely additive), indicating slight negative interaction — the two features share some redundancy via their proxy correlation with `deriv_zcr` and `std_mag`.

Conclusion: Both feature groups contribute positively and measurably to Phase C performance. Removing both `jerk` and `rolling_var_cv` simultaneously incurs the largest penalty (OOF F1: 0.7250 vs. 0.7361, **-0.0104**). `Jerk` is the stronger individual contributor: removing `jerk` alone costs -0.0067 OOF F1 (Config 2 vs. Config 4), while removing only `rolling_var_cv` costs -0.0026 (Config 3 vs. Config 4). Both feature groups provide genuine, additive discriminative value to the meta-learner and are retained in the v13 feature set.

Appendix: Pipeline Architecture Reference (v13)





Key Configuration Parameters (v13):

Parameter	Value
CNN: nb_filters	64 (hidden_dim = 256)
CNN: depth	6 SE-Inception blocks
CNN: bottleneck	32
CNN: SE reduction ratio	r = 16
CNN: dropout	0.4
Train: epochs (max)	80
Train: batch size	128
Train: learning rate	3×10^{-4} (AdamW)
Train: weight decay	1×10^{-3}
Train: LR schedule	CosineAnnealingLR (T_max=80, η_min= 1×10^{-6})
Train: patience	25
Train: focal γ	2.0
Train: label smoothing	ε = 0.1
Train: Mixup α	0.2
Meta: Optuna trials/fold	25
Meta: M_L2 range	[1.0, 5.0]

Parameter	Value
Meta: LGB n_estimators	1000
Meta: LGB learning_rate	0.05
Meta: LGB subsample	0.8